Exploratory Data Analysis and Visualization

Assignment II

Exploratory Data Analysis and Student Mathematics Performance Prediction Using Deep Learning

- Project Title: Exploratory Data Analysis and Student Mathematics Performance Prediction Using Deep LearningCourse: U21ADP05 - Exploratory Data Analysis and Visualization (Assignment II)
- Student Name: PRARTHANA S
- Roll No.: 24AD081
- Class: III - AD - B
- Department: Artificial Intelligence and Data Science
- Institution: KPR Institute of Engineering and Technology
- Year / Semester / Batch: III Year / V Semester / 2024-2028
- Course Coordinator: Ms. Revathi
- Date of Submission: 14/10/2026
- GitHub Repository: https://github.com/Prarthana2007/student_performance

1. ABSTRACT

This project analyzes student Mathematics performance using Exploratory Data Analysis (EDA), statistical methods, visualization, and Deep Learning. The UCI Student Performance dataset is used to study academic, demographic, family, and behavioural factors. Data preprocessing includes missing-value checks, duplicate removal, outlier detection, categorical encoding, and numerical standardization. Histograms, boxplots, scatter plots, and heatmaps are used to identify patterns and relationships. An MLP neural network is developed to classify students as pass or fail based on their academic and personal attributes. Model performance is evaluated using accuracy, a confusion matrix, and ROC-AUC. The findings may help educational institutions understand student performance patterns and explore academic support strategies.

2. INTRODUCTION AND OBJECTIVES

2.1 Introduction

Academic performance analysis helps identify factors associated with student achievement. EDA summarizes data and reveals patterns before Deep Learning is applied.

2.2 Problem Statement

This project investigates student Mathematics grades and examines whether selected student attributes can help classify pass/fail outcomes.

2.3 Aim

To analyze student Mathematics performance using EDA, visualization, statistical analysis, and Deep Learning.

2.4 Objectives

1. Understand and preprocess the dataset.
2. Analyze missing values, duplicates, and outliers.
3. Calculate descriptive statistics.
4. Create univariate, bivariate, and multivariate visualizations.
5. Encode categorical variables and standardize numerical features.
6. Split data into training, validation, and testing sets.
7. Build an MLP classification model.
8. Evaluate model performance using suitable metrics.
9. Interpret findings and identify future improvements.

2.5 Real-World Application

The analysis can help educational institutions investigate academic patterns and explore student-support strategies. Predictions should not be used as the sole basis for decisions about individual students.

3. DATASET DESCRIPTION

3.1 Dataset Source

- Dataset: Student Performance – Mathematics (student-mat.csv)
- Source: UCI Machine Learning Repository
- Link: https://archive.ics.uci.edu/dataset/320/student+performance
- Format: Semicolon-separated CSV

3.2 Dataset Size

Description	Value
Records	395
Original columns	33
Target	Pass
Passing threshold	G3 ≥ 10

3.3 Feature Description

The dataset includes demographic details, family background, school support, study habits, attendance, health, social factors, and academic grades.

Important features include age, studytime, failures, absences, G1, G2, and G3.

- G1: First-period grade
- G2: Second-period grade
- G3: Final Mathematics grade

3.4 Target Variable

A binary target, Pass, is created from G3.

- Pass = 1: G3 ≥ 10
- Pass = 0: G3 < 10

The original G3 column is excluded from the input features to prevent target leakage.

3.5 Statistical Description

Descriptive statistics such as mean, median, standard deviation, minimum, maximum, and quartiles are calculated using df.describe().

Insert screenshot: Dataset shape and df.describe() output from Google Colab.

4. EDA AND PREPROCESSING

4.1 Data Loading

The dataset is loaded using Pandas with sep=";". The first five rows, column names, data types, and dataset dimensions are examined.

4.2 Missing Values

Missing values are identified using df.isnull().sum(). Numerical values are imputed using the median and categorical values using the most frequent category, where required.

4.3 Duplicate Records

Duplicate rows are identified using df.duplicated().sum() and removed using df.drop_duplicates().

4.4 Outlier Detection

The Interquartile Range (IQR) method is used to identify potential outliers: [IQR=Q3-Q1]

Values outside the IQR boundaries are investigated rather than automatically removed.

4.5 Data Cleaning

Column names, data types, missing values, and duplicate records are checked. The Pass target is created from G3.

4.6 Categorical Encoding

Categorical variables are converted into numerical features using One-Hot Encoding.

4.7 Numerical Standardization

StandardScaler standardizes numerical features using the training data's mean and standard deviation.

4.8 Data Splitting

The dataset is divided into training (70%), validation (15%), and testing (15%) sets using stratified sampling. A random seed of 42 ensures reproducibility.

5. DATA VISUALIZATION AND INTERPRETATION

5.1 Pass/Fail Distribution

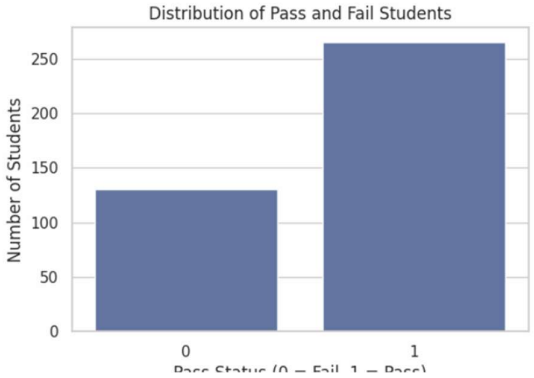


Image 1: Distribution of Pass and Fail Students

This bar chart shows the number of students in each class. It helps identify class imbalance between passing and failing students.

5.2 Final Grade Distribution

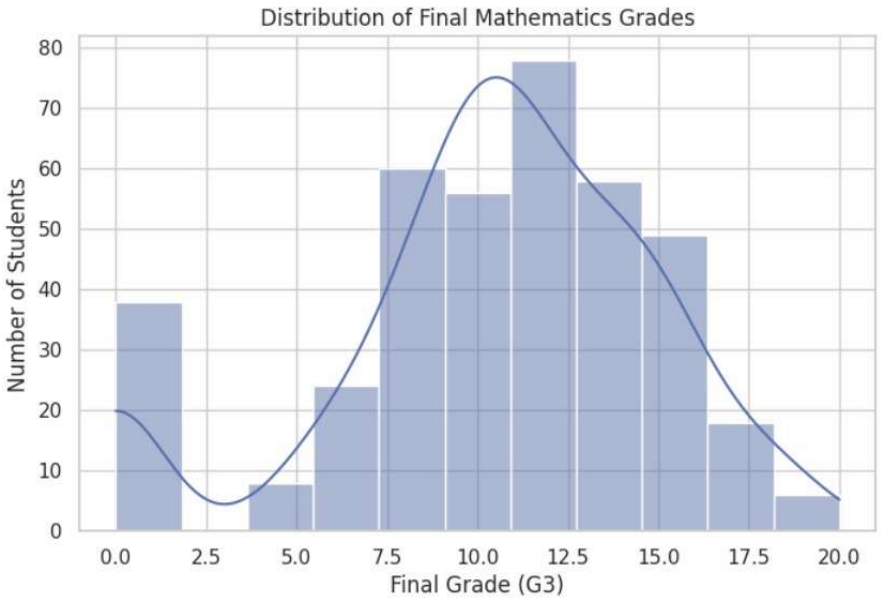


Image 2: Distribution of Final Mathematics Grades

The histogram and KDE curve show the distribution and concentration of final grades. They help identify skewness, grade concentration, and unusual values.

5.3 Study Time vs Final Grade

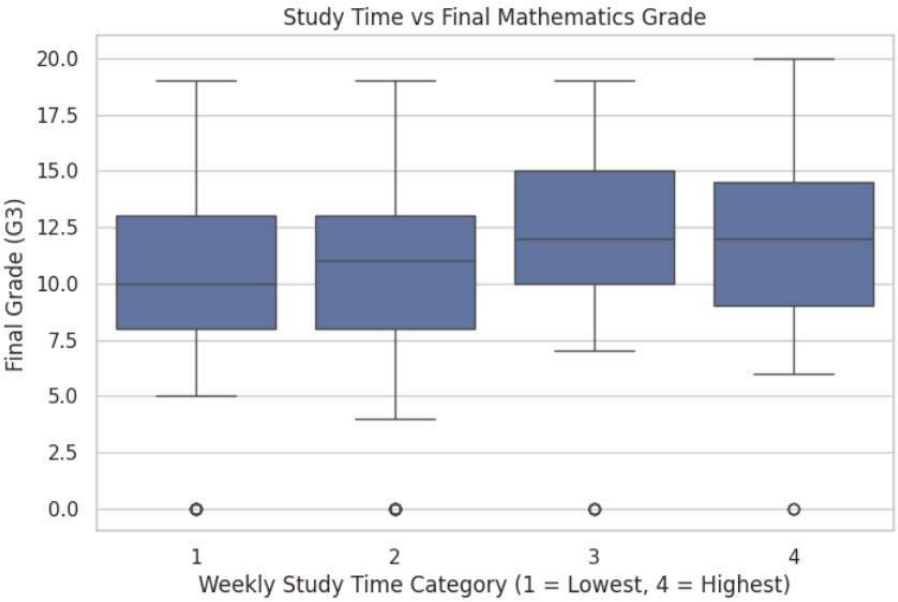


Image 3: Study Time vs Final Mathematics Grade

The boxplot compares final grades across study-time categories. It helps examine differences in median grades and grade variability.

5.4 Absences vs Final Grade

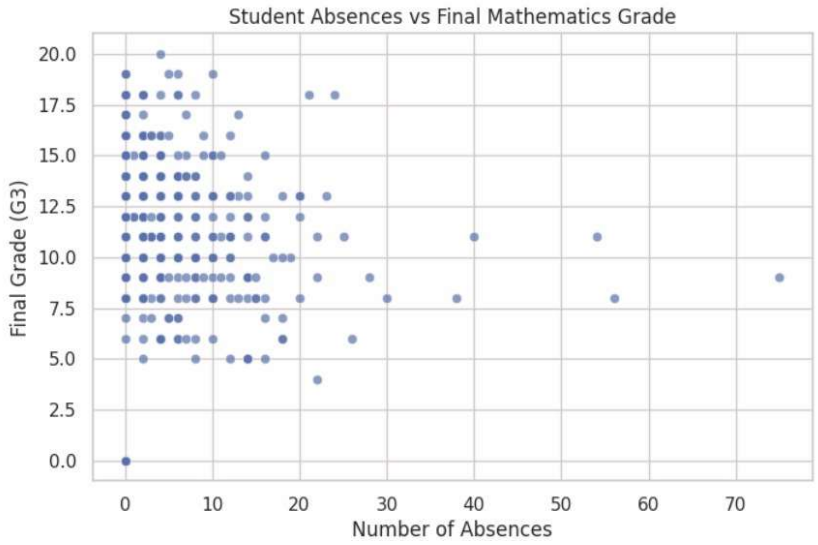


Image 4: Student Absences vs Final Mathematics Grade

The scatter plot investigates the relationship between absences and final grades. The spread of points indicates that absences alone may not explain academic performance.

5.5 Correlation Analysis

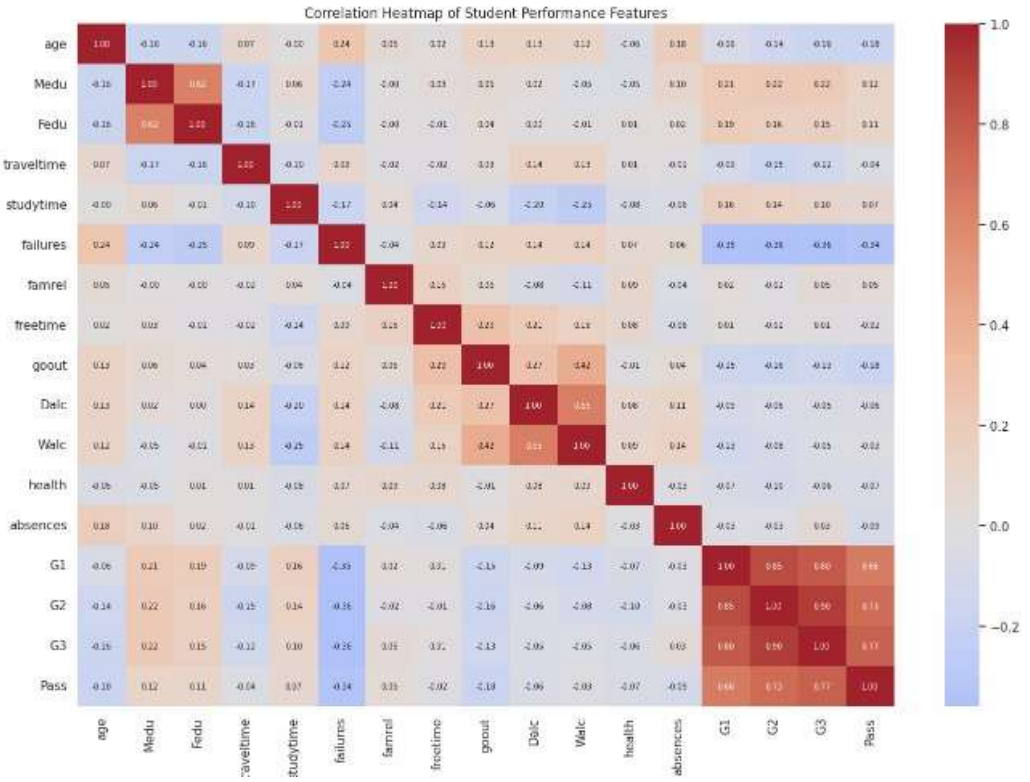


Image 5: Correlation Heatmap of Student Performance Features

Insert a correlation heatmap from Google Colab to examine relationships among numerical variables such as studytime, absences, G1, G2, and G3.

5.6 Pass/Fail Distribution

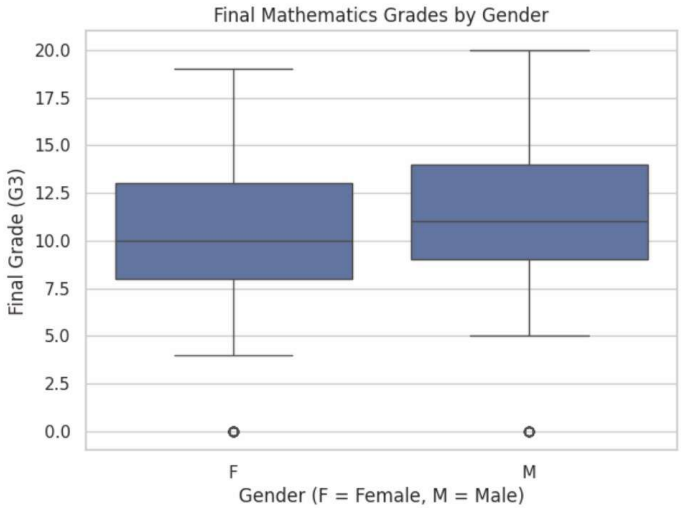


Image 6: Final Mathematics Grades by Gender

5.7 Mother's Education and Final Grade

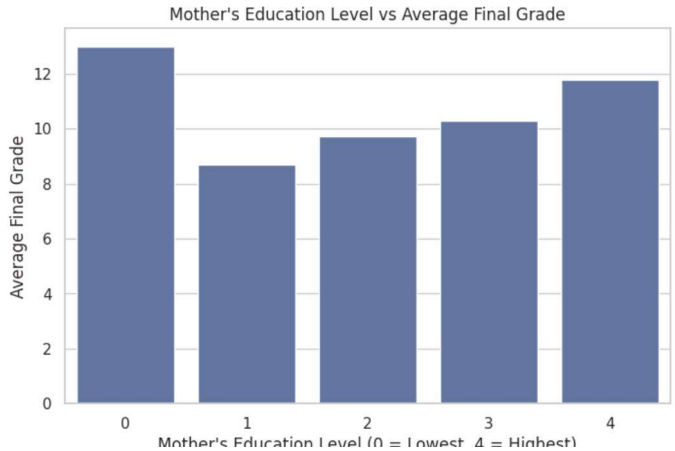


Image 7: Mother's Education Level vs Average Final Grade

5.8 Previous Grades vs Final Grade

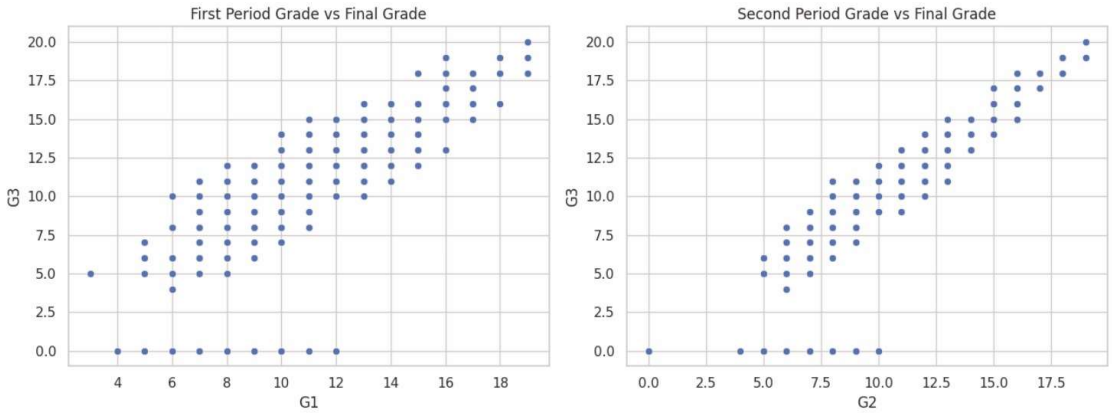


Image 8: First and Second Period Grades vs Final Grade

5.9 Overall Interpretation

The visualizations reveal the distribution of student grades and explore possible relationships between study time, attendance, and academic achievement. These relationships indicate patterns in the dataset but do not establish causation.

6. DEEP LEARNING MODEL

6.1 Model Architecture

A Multi-Layer Perceptron (MLP) neural network is developed using TensorFlow/Keras.

The model contains an input layer, hidden Dense layers with ReLU activation, dropout for regularization, and a sigmoid output layer for binary classification.

6.2 Training

The model is trained using the training set, while validation data monitors performance. Binary cross-entropy is used as the loss function, and Adam is used as the optimizer.

6.3 Evaluation

The test dataset is used to evaluate the trained model using accuracy, a confusion matrix, and ROC-AUC.

7. RESULT VISUALIZATION

1. Training and validation accuracy curves.

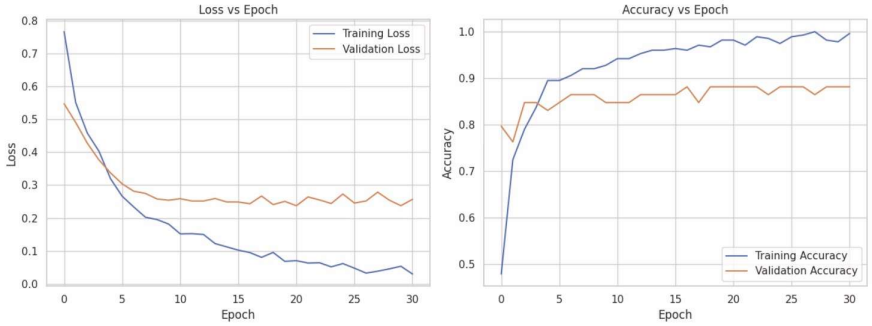


Image 9: Training and Validation Loss and Accuracy Curves

2. Training and validation loss curves.
3. Confusion matrix.

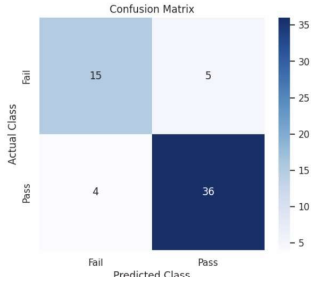


Image 10: Confusion Matrix for Student Pass Prediction

4. ROC curve with AUC score.

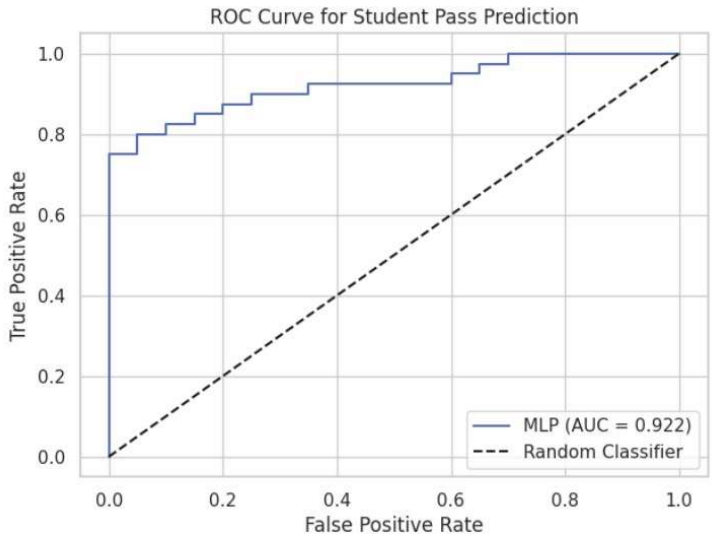


Image 11: ROC Curve for Student Pass Prediction

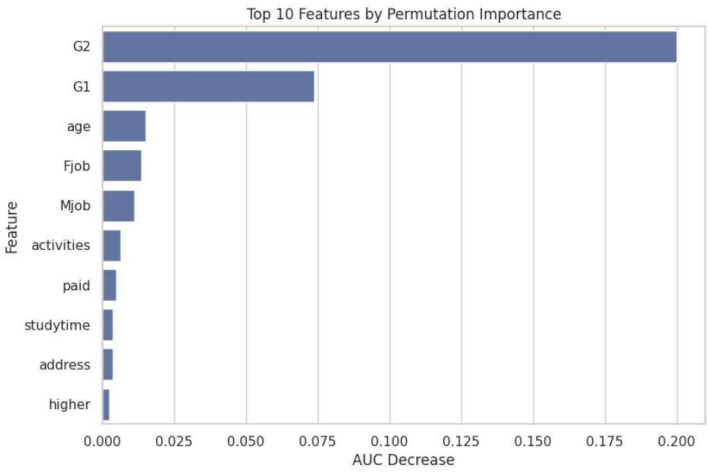


Image 12: Top 10 Features by Permutation Importance

These visualizations help assess predictive performance, classification errors, and possible overfitting.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project applies EDA, statistical analysis, visualization, and Deep Learning to student Mathematics performance data. The visualizations help identify grade distributions and explore relationships between academic and personal attributes. The MLP model provides an experimental approach to pass/fail classification.

8.2 Limitations

- The dataset contains a limited number of student records.
- Results may not generalize to all educational institutions.

- Predictions depend on the available features and data quality.

8.3 Future Scope

Future work can explore additional datasets, compare different classification models, tune hyperparameters, and investigate explainable AI techniques.

9. REFERENCES

1. Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. https://archive.ics.uci.edu/dataset/320/student+performance
2. McKinney, W. (2010). Data structures for statistical computing in Python.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python.
4. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning.

10. APPENDIX

Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
   accuracy_score, classification_report,
   confusion_matrix, roc_curve, roc_auc_score
)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import warnings
```

```
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
np.random.seed(42)
tf.random.set_seed(42)
print("All libraries imported successfully!")
print("TensorFlow version:", tf.__version__)
```

Upload your dataset

```
uploaded = files.upload()
file_name = next(
    name for name in uploaded.keys()
    if name.lower().endswith(".csv")
)
# Important: this dataset uses semicolons as separators
df = pd.read_csv(file_name, sep=";")
print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
display(df.head())
```

Understand the dataset

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nColumn names:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nDataset information:")
df.info()
print("\nFirst five rows:")
display(df.head())
print("\nStatistical summary:")
display(df.describe(include="all").T)
```

Checking missing values and duplicates

```
print("Missing values:")
display(df.isnull().sum().to_frame("Missing Count"))
print("\nTotal missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
# Remove exact duplicate rows
df = df.drop_duplicates().reset_index(drop=True)
# Remove unnecessary spaces from column names
df.columns = df.columns.str.strip()
print("\nShape after duplicate removal:", df.shape)
```

Outlier analysis

```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
outlier_summary = []
for col in numeric_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    outlier_summary.append({
        "Feature": col,
        "Lower Bound": lower,
        "Upper Bound": upper,
        "Potential Outliers": count
    })
display(pd.DataFrame(outlier_summary))
print("Outliers are identified, not automatically deleted.")
```

Create the target column

```
df["Pass"] = (df["G3"] >= 10).astype(int)
```

```
print("0 = Not Pass, 1 = Pass")
print(df["Pass"].value_counts())
print("\nClass distribution in percentage:")
print((df["Pass"].value_counts(normalize=True) * 100).round(2))
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Pass")
plt.title("Distribution of Pass and Fail Students")
plt.xlabel("Pass Status (0 = Fail, 1 = Pass)")
plt.ylabel("Number of Students")
plt.show()
```

Visualization 2: Study time and final grades

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="studytime", y="G3")
plt.title("Study Time vs Final Mathematics Grade")
plt.xlabel("Weekly Study Time Category (1 = Lowest, 4 = Highest)")
plt.ylabel("Final Grade (G3)")
plt.show()
display(df.groupby("studytime", observed=True)["G3"].agg(
    ["count", "mean", "median"]
).round(2))
```

Visualization 3: Absences and grades

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x="absences", y="G3", alpha=0.7)
plt.title("Student Absences vs Final Mathematics Grade")
plt.xlabel("Number of Absences")
plt.ylabel("Final Grade (G3)")
plt.show()
print(
    "Correlation between absences and final grade:",
    round(df["absences"].corr(df["G3"]), 3)
```

```
)
```

Visualization 4: Correlation heatmap

```
plt.figure(figsize=(14, 10))
correlation = df.select_dtypes(include=np.number).corr()
sns.heatmap(
    correlation,
    cmap="coolwarm",
    center=0,
    annot=True,
    fmt=".2f",
    annot_kws={"size": 7}
)
plt.title("Correlation Heatmap of Student Performance Features")
plt.tight_layout()
plt.show()
```

Visualization 5: Gender and final grades

```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x="sex", y="G3")
plt.title("Final Mathematics Grades by Gender")
plt.xlabel("Gender (F = Female, M = Male)")
plt.ylabel("Final Grade (G3)")
plt.show()
display(df.groupby("sex", observed=True)["G3"].agg(
    ["count", "mean", "median"]
).round(2))
```

Visualization 6: Parental education and student performance

```
plt.figure(figsize=(8, 5))
sns.barplot(data=df, x="Medu", y="G3", errorbar=None)
plt.title("Mother's Education Level vs Average Final Grade")
plt.xlabel("Mother's Education Level (0 = Lowest, 4 = Highest)")
```

```
plt.ylabel("Average Final Grade")
plt.show()
display(df.groupby("Medu", observed=True)["G3"].mean().round(2))
```

Visualization 7: Previous grades versus final grades

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.scatterplot(data=df, x="G1", y="G3", ax=axes[0])
axes[0].set_title("First Period Grade vs Final Grade")
sns.scatterplot(data=df, x="G2", y="G3", ax=axes[1])
axes[1].set_title("Second Period Grade vs Final Grade")
plt.tight_layout()
plt.show()
print("G1 vs G3 correlation:", round(df["G1"].corr(df["G3"]), 3))
print("G2 vs G3 correlation:", round(df["G2"].corr(df["G3"]), 3))
```

Prepare input features and target

```
X = df.drop(columns=["G3", "Pass"])
y = df["Pass"]
print("Input features:", X.shape)
print("Target:", y.shape)
print("\nInput columns:")
print(X.columns.tolist())
```

Split into training, validation and test sets

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
```

```
    random_state=42,
    stratify=y_temp
)
print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)
```

Preprocess numerical and categorical features

```
numeric_features = X_train.select_dtypes(
    include=np.number
).columns.tolist()
categorical_features = X_train.select_dtypes(
    exclude=np.number
).columns.tolist()
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
# Fit preprocessing only on training data
X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed = preprocessor.transform(X_val)
X_test_processed = preprocessor.transform(X_test)
# Convert to dense arrays for TensorFlow
```

```
X_train_processed = np.asarray(
    X_train_processed, dtype=np.float32
)
X_val_processed = np.asarray(
    X_val_processed, dtype=np.float32
)
X_test_processed = np.asarray(
    X_test_processed, dtype=np.float32
)
y_train_np = y_train.to_numpy(dtype=np.float32)
y_val_np = y_val.to_numpy(dtype=np.float32)
y_test_np = y_test.to_numpy(dtype=np.float32)
print("Preprocessing completed!")
print("Processed training shape:", X_train_processed.shape)
```

Build and train the Deep Learning model

```
model = keras.Sequential([
    layers.Input(shape=(X_train_processed.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.20),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
model.summary()
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
```

```
    patience=10,
    restore_best_weights=True
)
history = model.fit(
    X_train_processed,
    y_train_np,
    validation_data=(X_val_processed, y_val_np),
    epochs=100,
    batch_size=16,
    callbacks=[early_stopping],
    verbose=1
)
```

Training and validation curves

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(history.history["loss"], label="Training Loss")
axes[0].plot(history.history["val_loss"], label="Validation Loss")
axes[0].set_title("Loss vs Epoch")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[1].plot(history.history["accuracy"], label="Training Accuracy")
axes[1].plot(history.history["val_accuracy"], label="Validation Accuracy")
axes[1].set_title("Accuracy vs Epoch")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
plt.tight_layout()
plt.show()
```

Confusion matrix

```
cm = confusion_matrix(y_test_np, test_predictions, labels=[0, 1])
```

```
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Fail", "Pass"],
    yticklabels=["Fail", "Pass"]
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

ROC curve

```
fpr, tpr, thresholds = roc_curve(
    y_test_np, test_probabilities
)
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label=f"MLP (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Random Classifier")
plt.title("ROC Curve for Student Pass Prediction")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()
```

Feature importance analysis

```
rng = np.random.default_rng(42)
importance_results = []
baseline_auc = roc_auc_score(y_test_np, test_probabilities)
for col in X_test.columns:
```

```
    X_shuffled = X_test.copy()
    X_shuffled[col] = rng.permutation(X_shuffled[col].to_numpy())
    X_shuffled_processed = preprocessor.transform(X_shuffled)
    X_shuffled_processed = np.asarray(
        X_shuffled_processed, dtype=np.float32
    )
    shuffled_probabilities = model.predict(
        X_shuffled_processed, verbose=0
    ).ravel()
    shuffled_auc = roc_auc_score(
        y_test_np, shuffled_probabilities
    )
    importance_results.append({
        "Feature": col,
        "AUC Decrease": baseline_auc - shuffled_auc
    })
importance_df = pd.DataFrame(importance_results)
importance_df = importance_df.sort_values(
    "AUC Decrease", ascending=False
)
display(importance_df.head(10))
plt.figure(figsize=(9, 6))
sns.barplot(
    data=importance_df.head(10),
    x="AUC Decrease",
    y="Feature"
)
plt.title("Top 10 Features by Permutation Importance")
plt.show()
```

Save your project outputs

```
df.to_csv("cleaned_student_mathematics.csv", index=False)
importance_df.to_csv("feature_importance.csv", index=False)
model.save("student_performance_mlp.keras")
print("Project outputs saved successfully!")
print("- cleaned_student_mathematics.csv")
print("- feature_importance.csv")
print("- student_performance_mlp.keras")
```